hashlock.

Security Audit

LayerOneX (DEX)

September 2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Audit Findings	14
Centralisation	58
Conclusion	59
Our Methodology	60
Disclaimers	62
About Hashlock	63

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

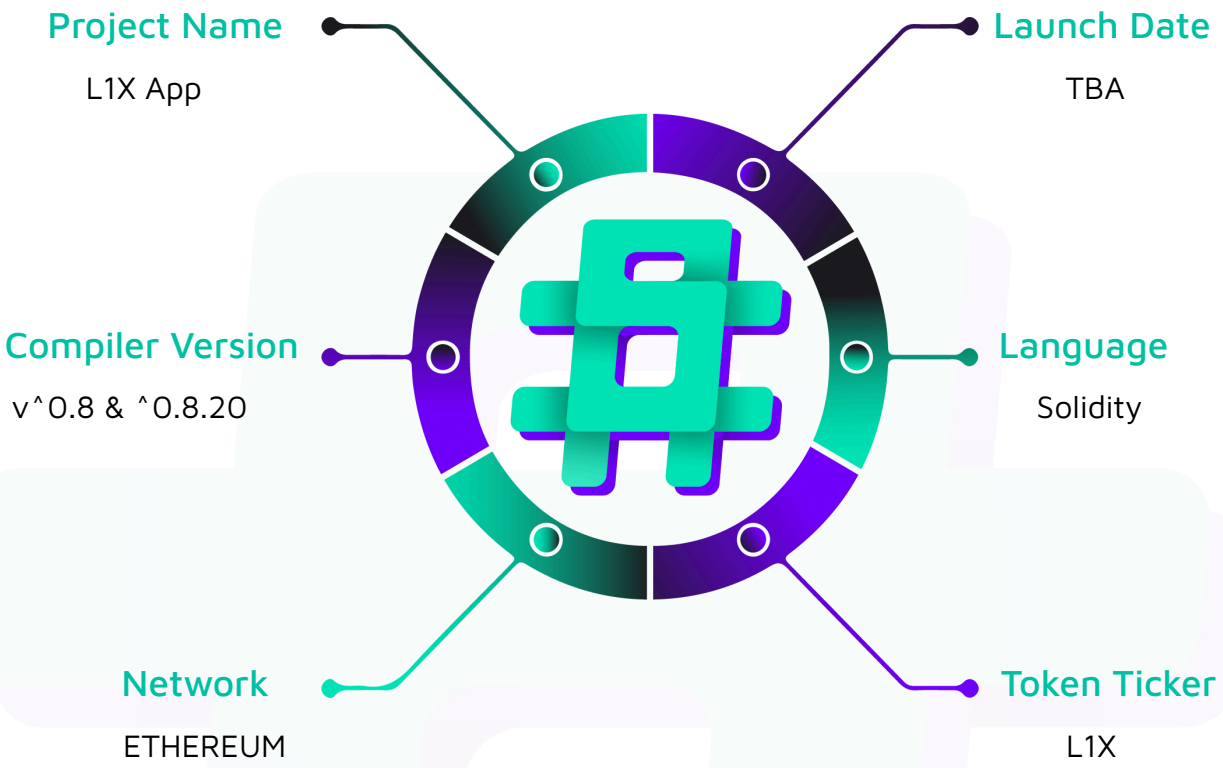
The LayerOneX team partnered with Hashlock to conduct a security audit of their smart contracts including StakeV2Contract.sol and SwapV2Contract.sol and L1XAppNFTSale.sol. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

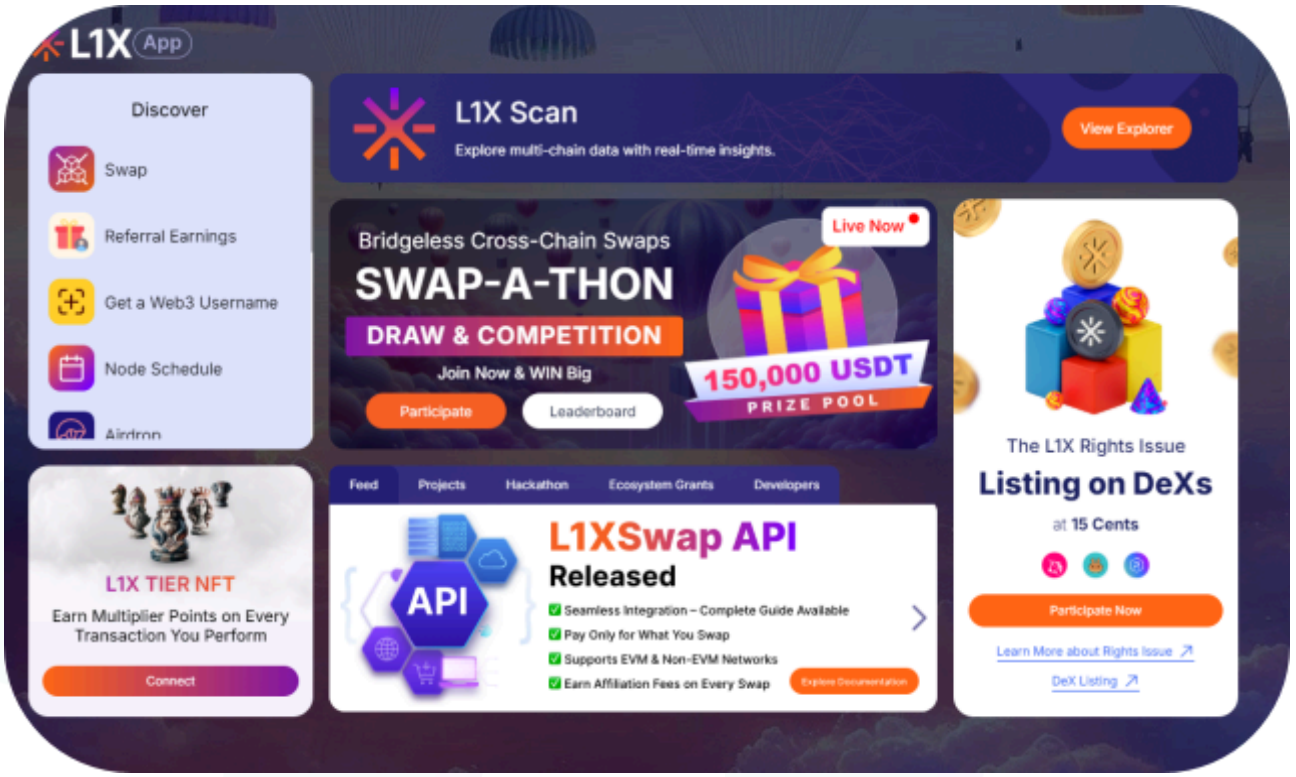
LayerOneX is a decentralized exchange (DEX) focused on Web3, providing a platform for trading cryptocurrencies and tokens without intermediaries. It offers features such as liquidity pools, staking, and decentralized governance, all built on blockchain technology. The platform emphasizes security, anonymity, and control for its users by leveraging the advantages of Web3 principles, such as self-custody and decentralized decision-making, making it ideal for users interested in decentralized finance (DeFi) and blockchain ecosystems.

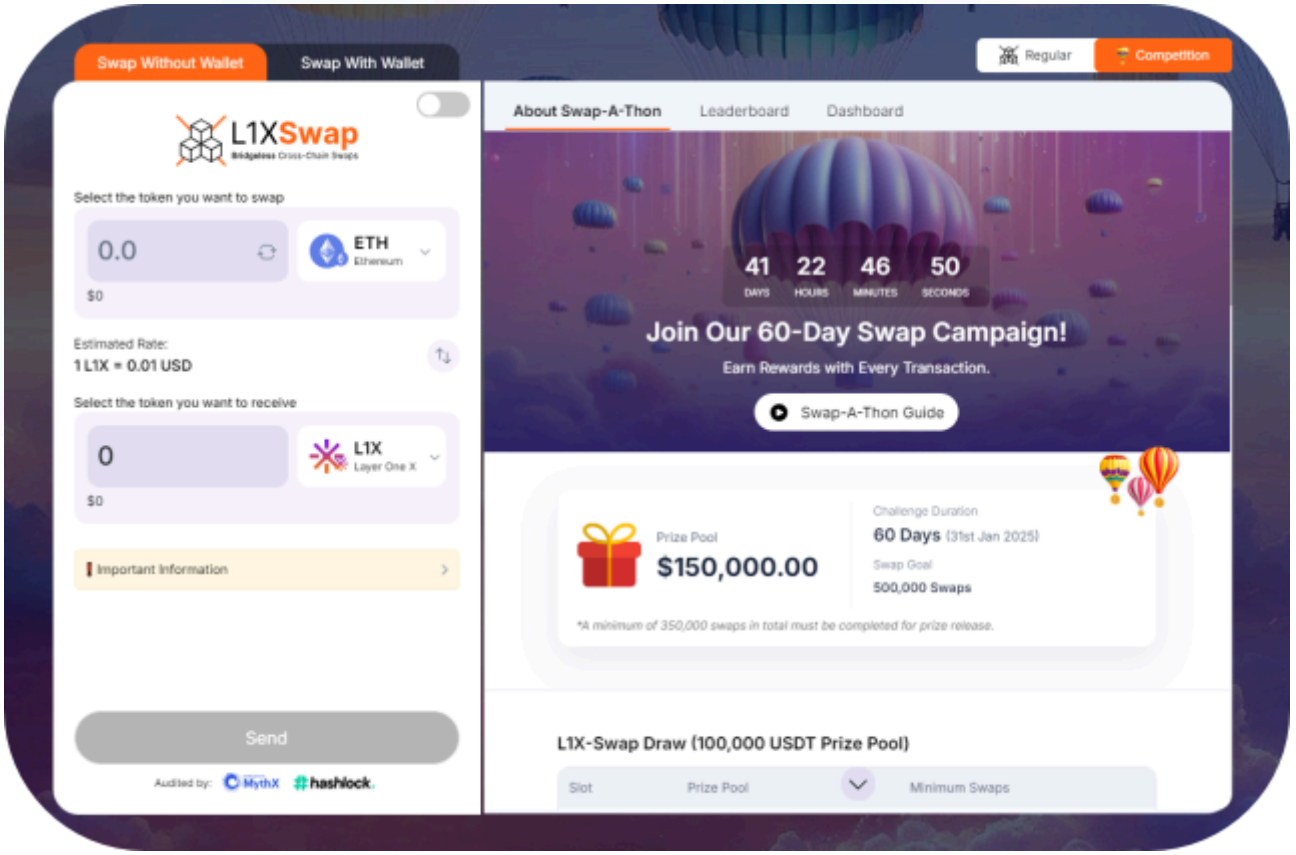
Project Name: L1X App
Compiler Version: ^0.8 & ^0.8.20
Website: https://l1xapp.com/
Logo:

Visualised Context:



Project Visuals:





Audit scope

We at Hashlock audited the solidity code within the LayerOneX project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**LayerOneX Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**September, 2024**
Contract 1	TreasuryContract.sol
Contract 2	XTalkGateway.sol
Contract 3	L1XAppNFTSale.sol
Contract 4	ZapContract.sol
Contract 5	SwapGasPriceV2Contract.sol
Contract 6	SwapV2Contract.sol
Contract 7	StakeV2Contract.sol
Contract 8	MultiSigWalletSingleBroadcast.sol

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 High severity vulnerabilities

8 Medium severity vulnerabilities

20 Low severity vulnerabilities

1 Gas Optimisation

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
TreasuryContract.sol - Allows Owner to: - Withdraw Native token - Withdraw ERC20 token - Rescue Total Native balance - Rescue ERC20 balance - Manage Authorised Sender, limits and receivers - Allows Authorised sender to: - Send ERC20 funds to recipients - Send Native token funds to recipients	**Contract achieves this functionality.**
L1XAppNFTSale.sol - Owner to: - Setup treasury address, share - Maintain ERC20 supported tokens - Maintain NFT price in USD based on month year - Maintain Authorised signers - Transfer funds to Treasury - User to: - Purchase NFT via native tokens - Purchase NFT via ERC20 tokens	**Contract achieves this functionality.**
XTalkGateway.sol - Allows Authorised Signer to: - Maintain Authorised Signers through voting and reaching majority - Entrypoint function to call external contract with data - Direct call function to make external call	**Contract achieves this functionality.**

ZapContract.sol - Allows Owners to: - Manage admin - Transfer funds to treasury - Allows Admins to: - Maintain source and destination fees - Maintain treasury address and share percentage - Maintain network and gateway information - Allows Users to: - Initiate a Zap request	**Contract achieves this functionality.**
SwapGasPriceV2Contract.sol Allows Authorised signer to: - Maintain network information - Information to compute gas cost for the network	**Contract achieves this functionality.**
SwapV2Contract.sol - Allows Owner to: - Maintain admin users - Transfer funds to treasury - Allows Admins to: - Configure treasury account and share percent - Update stake, gateway and gas fee contracts - Allows User to: - Transfer to stake - Initiate swap	**Contract achieves this functionality.**
StakeV2Contract.sol - Allows Owner to: - Manage authorised callers - Allows Authorised call to: - ability to pause deposit and withdrawal - Configure Withdrawal penalties and days - Deposit fees	**Contract achieves this functionality.**

- Withdraw funds to treasury - Asset configuration to list tokens supported for staking - Allows Users to: - Staking configured ERC20 Tokens - Staking native Token - Withdraw staked tokens	
MultiSigWalletSingleBroadcast.sol - Allows owners to: - Submit a transaction - Propose adding a new owner - Propose changing the required confirmations	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the LayerOneX project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the LayerOneX projects smart contracts code in the form of Zip file.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

High

[H-01] TreasuryContract#_sendFunds - The send limits authorised to the sender by the owner are not effective.

Description

The owner of `TreasuryContact` can authorise an account to be able send tokens with a limit. But, the limit authorised is not effective due to the following two reasons.

a) Available limit is not adjusted after each transfer

b) Limit is not configured on a token, hence ERC20 and native token are treated as equal

Vulnerability Details

The limit configured on each account intends to restrict the amount of tokens the account can transfer out from the `TreasuryContract` to recipients. Refer to `_setAuthorizedSender` function which configures a limit for the sender account. The limit is stored in `authorizedSenders` mapping.

```
function setAuthorizedSender(address sender, uint256 limit) external onlyOwner {

      authorizedSenders[sender] = limit; // @audit

      emit AuthorizationSet(sender, limit);

}
```

Now, refer to the `_sendFunds` and `_sendERC20Funds` functions which check if the current sender has a limit greater than or equal to the amount passed as a parameter. As long as the amount is less than or equal to the limit the funds can be sent out by the sender account.

```
function sendFunds(address payable recipient, uint256 amount) external authorizedOnly
nonReentrant {
```

```
  if(msg.sender != owner()) {

   require(authorizedSenders[msg.sender] >= amount, "Amount exceeds authorized limit");

   require(authorizedRecipients[msg.sender][recipient], "Recipient not authorized for this sender");

  }

  ....

  emit Withdrawn(recipient, amount);

}

function sendERC20Funds(address tokenAddress, address recipient, uint256 amount) external authorizedOnly nonReentrant {

 if(msg.sender != owner()) {

  require(authorizedSenders[msg.sender] >= amount, "Amount exceeds authorized limit");

 require(authorizedRecipients[msg.sender][recipient], "Recipient not authorized for this sender");

 }

 ...

 emit ERC20Withdrawn(recipient, tokenAddress, amount);

}
```

But, the vulnerability arises from the fact that, post transfer the limit is not adjusted. As a result, the sender can perform multiple transfers via separate transactions and send funds beyond the configured limit for the sender account by the owner.

Another aspect of vulnerability arises from how the limit is configured. Refer to the below code snippet where the amount being sent is compared with the configured limit.

```
 require(authorizedSenders[msg.sender] >= amount, "Amount exceeds authorized limit");
```

But, the value of ERC20, for example, DAI is very different from ETH. So, the value of 1000 ETH and 1000 DAI could mean very different for the protocol. This lack of consideration for value say in USD could result in a severe vulnerability.

Proof of Concept

Let's say, the owner configures Alice to be able to send 1000 as a limit. To perform this update, the owner calls `_setAuthorizedSender` function with Alice as the sender and limit as 1000. The owner also configures Bob to be a recipient for Alice by calling `setAuthorizedRecipient` function.

```
function setAuthorizedRecipient(address sender, address recipient) external onlyOwner {

     authorizedRecipients[sender][recipient] = true;

     emit RecipientAuthorized(sender, recipient);

 }
```

This enables Alice to send 1000 tokens and it can be Bob.

```
authorizedSenders[alice]=1000;
```

The problems arises when

a) Alice sends 1000 tokens to Bob, as the limit is not adjusted post transaction, Alice can send Bob 1000 tokens across 5 transactions and hence transfer 5000 tokens while the owner configured a limit of 1000 tokens.

b) Alice sends 1000 tokens of Ether instead of 1000 tokens for ERC20(USDT). The owners intent was to allow Alice to send only 1000 USDT, but instead Alice sends out 1000 Ether which is approx of 2,400,000 USDT in value.

The current implementation does not apply restrictions to prevent this kind of abuse by the authorised sender.

Impact

Authorised senders can send tokens way beyond the configured limit by the owner using multiple transactions or by choosing the type of token.

Recommendation

The limits should be configured at token level for each sender. Also, the limit should be reduced as the tokens are sent to the recipients by the sender.

Status

Resolved

[H-02] SwapGasPriceV2Contract#_bulkUpdateFees - Lack of proper validation in bulkUpdateFees could result in incorrect fee computation.

Description

The `_bulkUpdateFee` function lacks the validations of input arrays that could result in configuration of fees which will result in incorrect fee computation. The flaw arises due to how the input parameters can be passed, which could make this configuration ineffective.
The only line of defence is the access control restriction to authorised signatory with assumption that account will take care while setup, except for human error.

Vulnerability Details

In the below implementation of `_bulkUpdateFee`, there is a lack of validation for a range of values, which could result in a problem.

a) If `maxAmount` is less than `minAmount`

b) Overlapping ranges

c) Sorting the data set

```
function bulkUpdateFees(
        string memory network, uint256[] memory minAmounts,uint256[] memory maxAmounts,
        uint256[] memory constantFees,uint256[] memory feePercentages
    ) public onlyAuthorizedSigner returns (bool) {
        require(
            minAmounts.length == maxAmounts.length &&
            minAmounts.length == constantFees.length &&
            minAmounts.length == feePercentages.length,
```

```
            "Array lengths must match"
        );
        // Reset All fees for given network
        resetAllFees(network);
        for (uint256 i = 0; i < minAmounts.length; i++) {
            uint256 minAmount = minAmounts[i];
            uint256 maxAmount = maxAmounts[i];
            uint256 constantFee = constantFees[i];
            uint256 feePercentage = feePercentages[i];

                    feeStructures[network].push(FeeInfo(minAmount, maxAmount, constantFee,
feePercentage));
            feeIndex[network][minAmount][maxAmount] = feeStructures[network].length - 1;
        }
            emit BulkSlotFeeUpdated(network, minAmounts, maxAmounts, constantFees,
feePercentages);
    return true;
}
```

1. `MaxAmount` is less than `MinAmount`

If `maxAmount` is less than `minAmount`, then during the `getFeesForGivenAmountInUSD` function, due to the below condition will not be met and hence `feePercentage` and `constantFeeInUSD` will remain 0 after the loop.

```
for (uint256 i = 0; i < fees.length; i++) {
  if (assetValueInUSD >= fees[i].minAmount && assetValueInUSD <= fees[i].maxAmount) {
     feePercentage = fees[i].feePercentage;
     constantFeeInUSD = fees[i].constantFee;
     break;
  }
}
```

Example:

`assetValueInUSD = 100`

`fee[0] = {minAmount:120, maxAmount:85}`

`fee[1] = {minAmount:150, maxAmount:95}`

With this kind of configuration, the effective `feePercentage` and `constantFeeInUSD` will be 0 respectively.

2. Overlapping ranges:

`assetValueInUSD = 100`

`fee[0] = {minAmount:0, maxAmount:95}`

`fee[1] = {minAmount:90, maxAmount:150}`

`fee[2] = {minAmount:60, maxAmount:1000}`

As there is a potential for 100 to meet `fee(1)` and `fee(2)`, the question would be which is more appropriate. This is because there is no clear segregated range and sorting of data.

3. Sorting of data set is missing:

The arrays are not sorted and hence the fee applied could be random.

Impact

Fee application will not be correct

Recommendation

In order to fix these issues, the fee update logic should address all the below

a) `MaxAmount` cannot be less than `MinAmount` for an array index

b) The data set should be sorted in ascending order with discrete min and max ranges

c) There is no overlap of range across two array indexes.

Status

Resolved

[H-03] StakeV2Contract#_depositForNative - Instead of treasury share, the whole amount of Native tokens are transferred to SwapV2Contract.

Description

The `_depositForNative` function accepts Ether from User for Staking. After checking the conditions to accept deposit, the logic computes the treasury share based on the number of days. If the treasury share is greater than 0, then the intention is to transfer such a share amount to the Swap contract which has functionality to withdraw to the treasury address.

The bug is in the logic that performs the transfer of assets to swap contract. Instead of transferring the treasury share amount, the logic transfers the whole amount received from the caller.

Additionally, the `_depositForNative` function does not ensure if the number of days to stake is a valid configured amount by the authorised user.

Vulnerability Details

In the below `_depositForNative` function, the amount of Ether to be deposited is received as msg.value. If the durationInDays is configured in the contract, the treasury share is computed will be greater than 0 and will be stored in memory variable `swapDistribution.`

If the swapDistribution amount is greater than zero, the intention is to transfer the `swapDistribution` to Swap contract.

```
function depositForNative(
        uint256 durationInDays,
        address rewardAddress,
        string memory internalId
    ) external payable nonReentrant {
        require(isDepositPaused == false, "Deposit is paused");
         require(userStakesByTxId[internalId].amount == 0, "Stake with given internal ID already exist");
        require(rewardAddress != address(0), "Invalid Reward Address");
        require(SWAP_CONTRACT_ADDRESS != address(0), "Invalid Swap Address");
        uint256 amount = msg.value;
```

```
        require(amount > 0, "Amount must be greater than zero.");

uint256               totalDeposit               =               amount               +
assetDepositCapConfig[NATIVE_ASSET_ADDRESS].totalDeposited;

       require(totalDeposit  <=  assetDepositCapConfig[NATIVE_ASSET_ADDRESS].maxCap,  "Max
capping reached. Please try with lesser amount");

        // Handle Treeasury Share

Uint256                                  swapDistribution                                  =
calulatePercentAmount(assetConfig[NATIVE_ASSET_ADDRESS][durationInDays].treasuryPercentag
e, amount);

        if (swapDistribution > 0) {

         uint256 nativeAmount = msg.value;

         // Call the internal transfer function

         require(

                transferAsset(

                    NATIVE_ASSET_ADDRESS,

                    msg.sender,

                    SWAP_CONTRACT_ADDRESS,

                    swapDistribution,

                    nativeAmount

                ),

                "Transfer failed"

            );

        }

    …

}
```

The bug lies in how the transferAsset function works.

In the above code snippet, if the `swapDistribution` is greater than zero, then the `nativeAmount` variable is assigned with msg.value.

Hence, for the below `transferAsset` function:

`assetAddress = nominated Native Ether address`

`from = msg.sender`

`to = SWAP_CONTRACT_ADDRESS`

`amount = msg.value`

`nativeAmount = msg.value`

```
function transferAsset(
```

```
        address assetAddress,
        address from,
        address to,
        uint256 amount,
        uint256 nativeAmount
    ) internal returns (bool) {
        if (assetAddress == NATIVE_ASSET_ADDRESS) {
            // Ensure the correct amount of native asset is sent
            require(nativeAmount >= amount, "Incorrect amount of native asset sent");
            // Send native asset to the recipient
            (bool sent, ) = to.call{value: nativeAmount}("");// @audit
            require(sent, "Failed to send native asset");
            return true;
        } else {
            // ERC20 asset transfer
            IERC20 token = IERC20(assetAddress);
            require(token.balanceOf(from) >= amount, "Insufficient ERC20 balance");
               require(token.allowance(from, address(this)) >= amount, "Allowance is not
enough");
            // Use SafeERC20 to handle transfer
            token.safeTransferFrom(from, to, amount);
            return true;
        }
    }
```

As a result, the whole Ether deposited by the user represented in `msg.value` will be transferred to `SwapV2Contract`.

Proof of Concept

Alice deposits 10 Ether into the staking contract for 100 days, amount = 10
For 100 days, the share is 30% and hence 3 Ether, `swapDistribution = 3` `nativeAmount = 10`
Transfer treasury share to Swap contract using `transferAsset` function would be as below:

```
transferAsset(NATIVE_ASSET_ADDRESS,msg.sender,SWAP_CONTRACT_ADDRESS,3,10)
```

The actual transfer of ether in the `transferAsset` function is as below., which is actually transfer the whole 10 ether to the swap contract.

```
(bool sent, ) = to.call{value: nativeAmount}("");
```

Impact

The whole staked amount deposited by the user is transferred to the Swap contract.

Recommendation

Remove the call to `transferAsset` function. Transfer the treasury share by calling the low level call as below.

```
if (swapDistribution > 0) {
    (bool sent, ) = SWAP_CONTRACT_ADDRESS.call{value: swapDistribution}("");
    require(sent, "Failed to send native asset");
}
```

Status

Resolved

[H-04] StakeV2Contract#_durationInDays - Stake deposit functions for ERC20 and Ether does not validate for duration to stake, could result in permanent lock of user funds

Description

The `_depositForNative` function accepts native Ether for staking and likewise `_depositForERC20` function accepts deposit of different ERC20 tokens. The user has to specify the lock duration in a number of days, which is passed as parameter by the user. `_depositForNative` and `_depositForERC20` functions do not validate the stake duration as a validate and acceptable period for the protocol.

If a user specifies a long duration, say 100000 days, then the user will not be able to withdraw such funds during this lifetime. This risk arises due to lack of validation on acceptable time windows configured by the authorised user.

Vulnerability Details

In the staking contract, the `assetConfig` mapping maintains the apr and treasury percentage for each asset based on the staking time window. This mapping is maintained by the authorised user. Hence, only deposits for time windows that are configured in the mapping should be allowed.
If the user attempts to deposit for time window outside the configured periods should be rejected.

Proof of Concept

Lets say, for USDT, the below periods are configured by the authorised user.

`assetConfig[USDT][30] = {apr:10, treasuryPercentage:5}`
`assetConfig[USDT][60] = {apr:11, treasuryPercentage:6}`
`assetConfig[USDT][90] = {apr:12, treasuryPercentage:7}`
`assetConfig[USDT][120] = {apr:13, treasuryPercentage:8}`

Users staking USDT should be able to stake only for 30,60,90 or 120 days. For any other duration, the transaction should be reverted. If a user locks functions for 100000 days, then the user cannot withdraw functions due to the below function. Due to the below condition, user will not be able to withdraw funds.

```
function isStakeUnlocked(
        string memory internalId
    ) public view returns (bool) {
        Stake memory stake = userStakesByTxId[internalId];
        if (stake.startDate == 0) {
            return false;
        }
        uint256 elapsedTime = getCurrentTimestamp() - stake.startDate;
        if (elapsedTime >= stake.durationInDays * (1 days)) {
            return true;
```

```
        }
        return false;
    }
```

Impact

Letting users choose the time period for staking could result in lockup of funds.

Recommendation

The recommendation is to add the below validation to both deposit functions

```
require(assetConfig[assetAddress][durationInDays].treasuryPercentage!=0 &&
         assetConfig[assetAddress][durationInDays].apr!=0,"Asset is not configured");
```

Status

Resolved

[H-05] SwapV2Contract#__l1xReceive- Irrespective of the status of the swap on L1, the swap function is getting executed on the destination chain

Description

In the `_l1xReceive` function, the received message from L1 is decoded to understand the status of the transaction. If the status of the transaction is true, it is expected to complete the related swap on the destination chain.
But, in case the status was false, then only the status of the transaction should be updated for the swap. It would be incorrect to execute the sway function call.

Vulnerability Details

Refer to the code snippet from `_l1xReceive` function, where `L1Gateway` sends an encoded message with status of the transaction.
If the transaction status was not successful, then the swap should not be executed on the destination chain as well. But, in the below code snippet, the `executeSwap` function is called

irrespective of the status of transaction on L1.

```
function _l1xReceive(
        bytes32 globalTxId, bytes memory message) external nonReentrant onlyL1XGateway {
        (uint256 _amount, address _receiverAddress, address _assetAddress,
            string memory _internalId, bool _status, string memory _statusMessage
        ) = abi.decode(message,(uint256, address, address, string, bool, string)
            );
        require(
            getLength(swapStatus[_internalId].internalId) == 0,
            "Message already acknowledged"
        );
        // Check if swap already completed
        require(
            _status == true ||
                (_status == false &&
                    swapsInitiated[_internalId].sourceAmount != 0),
            "Invalid X-Talk execution"
        );

        uint256 _swapsCompleted;
        if (_status == true) {
            _swapsCompleted = SWAP_COMPLETED;
        } else {
            _swapsCompleted = SWAP_FAILED;
        }

        swapStatus[_internalId] = SwapExecutionData(
            globalTxId,_amount,_receiverAddress,_assetAddress,_internalId,
            _status,_statusMessage,_swapsCompleted
        );

        // Execute Swap
        executeSwap(_amount, _receiverAddress, _assetAddress);
```

Impact

The transaction status across the two chains gets out of sync and caller of the swap function could lose funds

Recommendation

`executeSwap` function should be called only if the transaction was successful in L1.

Status

Resolved

Medium

[M-01] L1XAppNFTSale#_purchaseViaStableERC20 - Should return any excess ERC20 received back to the caller, otherwise the caller could end up paying excess tokens.

Description

The `_purchaseViaStableERC20` function accepts the estimated amount of ERC20 tokens required to complete the purchase of NFT. But, the actual amount needed is computed in the function using the month Id for price and the quantity of NFT being purchased.

There is a flaw in the logic for computation of treasury share, which should be based on the computed value and should not be on estimated amount, which could be higher to prevent reverting of the transaction.

Vulnerability Details

Refer to the below `_purchaseViaStableERC20` function's code snippet, where `totalAmountInAssetValue` is passed as input parameter by the caller.

Lets say, Alice is purchasing 3 NFTs with each NFT being 100 USDT for the current month.
Alice passes `totalAmountInAssetValue` as 350 USDT to cover treasury share.

The function logic will read the NFT price for the `monthId` and scale it which is stored in `totalAmountCalculated`.For the above example, `totalAmountCalculated` would be 100*3 (ignoring the scaling should be 300.

Hence the treasury share should be calculated on 300 and not 350 as implemented in the logic. Assuming 10% as treasury share, 300 * 10/100 = 30 should be transferred to the treasury account instead of 35 as implemented in the current logic.

Any excess tokens should be returned back to the caller:

```
function purchaseViaStableERC20( // @audit, does this have to be payable
        string[] memory arrUserName,
        address investmentAssetAddress,
        address NFTReceiverAddress,
        address rewardReceiverAddress,
        uint256 qty,
        uint256 totalAmountInAssetValue,
        uint256 monthId
    )
     external nonReentrant payable
    {
      …
     // Scale Amount for USD value from 8 decimals to given asset decimal places
      uint256 totalAmountCalculated = scaleAmountDecimal(NFTUSDPriceByMonthId[monthId] * qty, USD_DECIMAL , IERC20Extended(investmentAssetAddress).decimals());
     require(totalAmountInAssetValue >= totalAmountCalculated,"totalAmountInAssetValue is less than calculated value");
    // Transfer All funds to this contract
                IERC20(investmentAssetAddress).safeTransferFrom(msg.sender,address(this), totalAmountInAssetValue);
        // Calculate Treasury Share of Source Amount ERC20
uint256                          treasuryShareForSourceAmount                          = calculatePercentAmount(treasuryShareForSourceAmountPercent,totalAmountInAssetValue);
}
```

Impact

Callers purchasing the NFT could end up paying excess tokens to the protocol, essentially leading to losses for users.

Recommendation

Return excess tokens back to the caller.

Status

Resolved

[M-02] L1XAppNFTSale#_purchaseViaNative - Should return any excess native tokens received back to the caller, otherwise the caller could end up paying excess tokens.

Description

The `_purchaseViaNative` function accepts native token Ether via the msg.value to cover the funds needed to purchase the NFTs. The actual amount payable towards the purchase of NFTs is based on the quantity of NFTs purchases and Ether price in USD based on the price configured for the monthId.

The treasury share should be computed based on the actual amount payable and not on the msg.value. The flaw in the logic to use msg.value for computing treasury shares will result in buyers paying a higher amount which negatively impacts the user.

Vulnerability Details

In `_purchaseViaNative` function, msg.value which represents the amount of native token received is expected to be higher than `totalAmountInAssetValueCalculated` calculated based on the number of NFT tokens and applicable price.
The flaw is in the logic of computing the treasury share on msg.value instead of `totalAmountInAssetValueCalculated` which is the correct value of the transaction.
As a result of this flaw, buyers end up paying more ether than actually required for the purchase of NFTs.

```
function purchaseViaNative(string[] memory arrUserName,
        address NFTReceiverAddress,
        address rewardReceiverAddress,
        uint256 qty,
        uint256 monthId,
        uint80 roundId
    )  external nonReentrant payable
    {
       …
uint256                          unitAmountInAssetValueCalculated                          =
getSpecifictNativeQuoteByQty(1,monthId,roundId);
```

```
Uint256                          totalAmountInAssetValueCalculated                          =
getSpecifictNativeQuoteByQty(qty,monthId,roundId);

require(msg.value  >=  totalAmountInAssetValueCalculated,"totalAmountInAssetValue  is  less
than calculated value");

// Calculate Treasury Share of Source Amount ERC20

Uint256                            treasuryShareForSourceAmount                           =
calculatePercentAmount(treasuryShareForSourceAmountPercent,msg.value);

        if (treasuryShareForSourceAmount > 0) {

            // Transfer Treasury Share to Treasury

            payable(treasuryContract).transfer(treasuryShareForSourceAmount);

     }

   …

   }
```

Impact

Callers purchasing the NFT could end up paying excess ether to the protocol, essentially leading to losses for users.

Recommendation

Return excess ether back to the caller.

Status

Resolved

[M-03] L1XAppNFTSale#_authorizedSigner - authorised signer framework is implemented in L1XAppNFTSale, but was not used.

Description

`L1XAppNFTSale` contract implements the authorised signer framework in the contract, but was not used in the contract. The concern is around the overlooking for intended restrictions on access of the functionality which was not implemented.

Vulnerability Details

`L1XAppNFTSale` contract has a mapping for `authorizedSigner`, a modifier to check if the current user is `authorizedSigner` or not. The contract has two owner enabled functions to maintain the `authorizedSigner` mapping.
But, the contract does not use the authorised signer anywhere in the contract.

```
mapping(address => bool) public authorizedSigner;


modifier onlyAuthorizedSigner() {
  require(authorizedSigner[msg.sender] == true, "Not Authorized Signer");
  _;
}
// Function to add a new authorized signer
function addAuthorizedSigner(address signer) external onlyOwner {
  require(signer != address(0), "Signer is the zero address");
  require(!authorizedSigner[signer], "Signer is already authorized");
  authorizedSigner[signer] = true;
  emit SignerAdded(signer);
}
// Function to remove an authorized signer
function removeAuthorizedSigner(address signer) external onlyOwner {
  require(authorizedSigner[signer], "Signer is not authorized");
  authorizedSigner[signer] = false;
  emit SignerRemoved(signer);
}
```

Impact

Missing intended restriction

Recommendation

Review and implemented the restriction for authorised signer where applicable

Status

Resolved

[M-04] ZapContract#_initiateZap - initiateZap should return back any excess ether left after processing the transaction.

Description

The `_initiateZap` function initiates a zap request, as a part of which it accepts tokens from the caller. The tokens can be ERC20 or native Ethereum tokens. Hence, the validation logic should check if ERC20 is the mode of receiving tokens, msg.value should be 0 so to prevent any ether sent by mistake.

Users initiating the `_initiateZap` function could be sending excess ether to cover the transaction cost including fees and to prevent reverts. The function logic should compute and refund any excess ether received back to the caller.

Vulnerability Details

Refer to the below check in the `initiateZap` function, where msg.value could be greater than `_sourceAmount` + `_sourceFee`. If it was exactly equal, the transaction will go through. That means, any ether above `_sourceAmount` + `_sourceFee` is excess and should be returned back to the caller.

```
function _initiateZap(...) external {

 …

    require(msg.value>=(_sourceAmount    +    _sourceFee),"Insufficient    Source    Amount"
);

 …

}
```

The current implementation does not refund the excess ether to the caller.

Impact

Users may send excess ether to prevent reverts, but the function logic does not return any excess ether back to the user resulting in charging excess ether from the user.

Recommendation

Return excess Ether back to the user

Status

Resolved

[M-05] StakeV2Contract#_transferToTreasury - uses a low level call function to move native tokens, but does not check returned boolean for success or failure of the transaction.

Description

The `_transferToTreasury` function uses the correct methodology to move native ether by calling the low level call function. This function is much more flexible in terms of gas limits and hence it is the recommended way to move native tokens from one account to another. But, in order to confirm if the transaction was successful or failure, it is critical that the caller checks the returned boolean flag. Ignoring the return boolean flag will result in inaccurate accounting.

Vulnerability Details

```
function transferToTreasury(address tokenAddress,uint256 amount)external
onlyAuthorizedCaller {
  require(treasuryContract != address(0), "Treasury address is not set");
  if (tokenAddress == NATIVE_ASSET_ADDRESS) {
    // Send Native to Treasury with reentrancy Protection for Native
    (bool sent, bytes memory data) = treasuryContract.call{value: amount}("");
  } else {
```

In the above code snippet, the call returns bool sent, which should be checked to revert in case the transaction failed.

```
(bool sent, bytes memory data) = treasuryContract.call{value: amount}("");
require(sent,"Transfer failed");
```

The require statement above is critical to ensure that transaction will revert incase of failure to move native tokens.

Impact

Results in incorrect accounting, if boolean is ignored

Recommendation

Add validation for the flag as above

Status

Resolved

[M-06] SwapV2Contract#_initiateSwap - initiateswap should return back any excess ether left after processing the transaction.

Description

The `_initiateSwap` function initiates a swap request, as a part of which it accepts tokens from the caller. The tokens can be ERC20 or native Ethereum tokens. Hence, the validation logic should check if ERC20 is the mode of receiving tokens, msg.value should be 0 so to prevent any ether sent by mistake.

Users initiating the `_initiateswap` function could be sending excess ether to cover the transaction cost including fees and to prevent reverts. The function logic should compute and refund any excess ether received back to the caller.

Vulnerability Details

Refer to the below check in the `initiateswap` function, where msg.value could be greater than `_sourceAmount` + `_sourceFee`. If it was exactly equal, the transaction will go through.
That means, any ether above `_sourceAmount` + `_sourceFee` is excess and should be returned back to the caller.

```
function _initiateSwap(...) external {
 …
 require(
                msg.value >= _sourceAmount + _sourceNativeFee,
                "Insufficient Fee Provided"
            );
 …
```

```
}
```

The current implementation does not refund the excess ether to the caller.

Impact

Users may send excess ether to prevent reverts, but the function logic does not return any excess ether back to the user resulting in charging excess ether from the user.

Recommendation

Return excess Ether back to the user. Also, when input tokens are ERC20, msg.value should be validated for 0.

Status

Resolved

[M-07] StakeV2Contract#_transferAssetForWithdrawal - uses a low level call function to move native tokens, but does not check returned boolean for success or failure of the transaction.

Description

The `_transferAssetForWithdrawal` function uses the correct methodology to move native ether by calling the low level call function. This function is much more flexible in terms of gas limits and hence it is the recommended way to move native tokens from one account to another. But, in order to confirm if the transaction was successful or failure, it is critical that the caller checks the returned boolean flag. Ignoring the return boolean flag will result in inaccurate accounting.

Vulnerability Details

```
function transferAssetForWithdrawal(
        address assetAddress,
        address from,
        address to,
        uint256 amount
```

```
    ) internal returns (bool) {
     if (assetAddress == NATIVE_ASSET_ADDRESS) {
     require(from.balance >= amount, "Insufficient Native balance");
     // Send Native to Treasury with reentrancy Protection for Native
     (bool sent, ) = to.call{value: amount}("");
     return sent;
    } else {
          require(IERC20(assetAddress).balanceOf(from)  >=  amount,  "Insufficient  ERC20
balance");
     // Send Native to Treasury with reentrancy Protection for ERC20
     IERC20(assetAddress).safeTransfer(to, amount);
     return true;
   }
}
```

In the above code snippet, the call returns bool sent, which should be checked to revert in case the transaction failed.

```
 (bool sent,) = to.call{value: amount}("");
 require(sent,"Transfer failed");
```

The require statement above is critical to ensure that transaction will revert incase of failure to move native tokens.

Impact

Results in Incorrect accounting, if boolean is ignored

Recommendation

Add validation for the flag as above

Status

Resolved

[M-08] StakeV2Contract#_validateRequiredAmount - is vulnerable to arithmetic underflow exception

Description

The `_validateRequiredAmount` function is vulnerable to arithmetic underflow exceptions arising due to the configured values by authorised signers. The `validateRequiredAmount` computes `depositFee` and `amountAfterPenalty` , the percentage of these are computed by the authorised signers. The vulnerability will occur when `depositFee` is greater than `amountAfterPenalty`.

The possibility of vulnerability exists because the deposit fee percentage can be configured upto 100% of the staked amount.

Vulnerability Details

```
function validateRequiredAmount(string memory internalId) public view returns (
        ...) {
        address _tokenAddress = userStakesByTxId[internalId].assetAddress;
        uint256 _stakeAmount = userStakesByTxId[internalId].amount;// 100
        uint256 _depositFeePercent = userStakesByTxId[internalId].depositFeePercent;
        bool _isRedeemed = userStakesByTxId[internalId].isRedemed;
        require(_isRedeemed == false, "Stake is already withdrawn");
        (
            uint256 amountAfterPenalty,
            uint256 penaltyAmount
        ) = calculateAmountLeftAfterPenalty(internalId);
        uint256 _depositFee = calulatePercentAmount(_depositFeePercent, _stakeAmount);
        uint256 _requiredAmount = amountAfterPenalty - _depositFee;
        ...
}
```

Example: Staked amount is 10 Ether

Amount after penalty applicable is 0.9 Ether

Deposit fee is configured as 100%, hence it would be 10 ether

```
uint256 _requiredAmount = amountAfterPenalty - _depositFee;
                        = (10 - 0.9) - 10
                        = 9.1 - 10
```

The above computation will result in arithmetic error.

Impact

This will prevent withdrawals as the transactions will revert

Recommendation

Review the deposit fee percentage and restrict acceptable values to a valid range.

Status

Resolved

Low

[L-01] Contracts - Outdated version of Solidity and also multiple versions are in use.

Description

The project uses `pragma solidity ^0.8.0` and `^0.8.20` across different files.

Recommendation

Use `pragma solidity 0.8.23`. It is recommended to use a single version across the project for files that are not yet deployed. Also, lock the solidity version to a specific version.

Status

Acknowledged

[L-02]TreasuryContract#_withdraw - Uses Legacy transfer() function to move native token to recipient address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below:

```
Function withdraw(address payable recipient, uint256 amount) external onlyOwner nonReentrant {

        require(address(this).balance >= amount, "Insufficient balance");
```

```
        (bool success) = recipient.call{value:amount}("");
        require(success,"withdrawal failed");
        emit Withdrawn(recipient, amount);
}
```

Status

Resolved

[L-03]TreasuryContract#_rescureAllBalanceForNative - Uses Legacy transfer() function to move native token to recipient address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below:

```
function rescureAllBalanceForNative() external onlyOwner {
    uint256 balance = address(this).balance;
    (bool success) = msg.sender.call{value:balance}("");
    require(success,"rescue of native balance failed");
    emit Withdrawn(owner(), balance);
}
```

Status

Resolved

[L-04]TreasuryContract#_sendFunds - Uses Legacy transfer() function to move native token to recipient address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below:

```
function sendFunds(address payable recipient, uint256 amount) external authorizedOnly nonReentrant {
  if(msg.sender != owner()) {
   require(authorizedSenders[msg.sender] >= amount, "Amount exceeds authorized limit");
     require(authorizedRecipients[msg.sender][recipient], "Recipient not authorized for this sender");
  }
  (bool success) = recipient.call{value:amount}("");
  require(success,"sendFunds failed");
  emit Withdrawn(recipient, amount);
}
```

Status

Resolved

[L-05]L1XAppNFTSale#_NFTInvestment - NFTInvestment structure has some commented variables.

Description

Refer to the below declaration for `NFTInvestment`. Fields like qty, `totalAmountInUSD` and `totalAmountInAssetValue` are commented out. Remove these fields from the structure declaration.

```
struct NFTInvestment {
       string userName;
       address investmentAssetAddress;
       uint8 investmentAssetDecimals;
       address NFTReceiverAddress;
       address rewardReceiverAddress;
       // uint256 qty;
       uint256 unitPriceInUSD;
       uint256 unitPriceInAssetValue;
       // uint256 totalAmountInUSD;
       // uint256 totalAmountInAssetValue;
       uint256 timestamp;
       uint256 monthId;
}
```

Recommendation

Remove these fields from the structure declaration.

Status

Resolved

[L-06]L1XAppNFTSale#_transferToTreasury - Uses Legacy transfer() function to move native token to treasury contract address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below.

```
function transferToTreasury(address tokenAddress, uint256 amount) external onlyOwner {
        require(treasuryContract != address(0), "Treasury address is not set");
        if(tokenAddress == NATIVE_ASSET_ADDRESS) {
            // Send Native to Treasury with reentrancy Protection for Native
            (bool success) = treasuryContract.call{value:amount}("");
            require(success,"transfer to treasury failed");
        }
        else
        {
            // Send Native to Treasury with reentrancy Protection for ERC20
            IERC20(tokenAddress).safeTransfer(treasuryContract, amount);
        }
    }
```

Status

Resolved

[L-07]L1XAppNFTSale#_transferToTreasury - transferToTreasury function should alway only ERC20 tokens that are listed as supported by the owner

Description

`_transferToTreasury()` function should allow transfer of ERC20 tokens only for token addresses that are listed as supported in the contract by the owner. In the current implementation, such validation is missing.

Recommendation

The recommendation is to add validation in the ERC20 transfer flow as below so that any attempt to transfer unsupported tokens will revert.

```
function transferToTreasury(address tokenAddress, uint256 amount) external onlyOwner {
        require(treasuryContract != address(0), "Treasury address is not set");
        if(tokenAddress == NATIVE_ASSET_ADDRESS) {
          …
        }
        else
        {
             require(supportedERC20[tokenAddress],"Token address is not supported");
            // Send Native to Treasury with reentrancy Protection for ERC20
            IERC20(tokenAddress).safeTransfer(treasuryContract, amount);
        }
    }
```

Status

Resolved

[L-08]L1XAppNFTSale#_purchaseViaStableERC20 - purchaseViaStableERC20 function operates on ERC20 tokens and hence does not need payable operator

Description

`_purchaseViaStableERC20` function operates only on ERC20 tokens and does not deal with native token, ETH. If the caller of `_purchaseViaStableERC20` passed ether by mistake, the ether will be deposited in the `L1XAppNFTSale` contract without accounting in favour of the caller. Payable operator is needed only for functions that accept native ether token.

Recommendation

Remove the payable operator from `_purchaseViaStableERC20` function.

Status

Resolved

[L-09]L1XAppNFTSale#_purchaseViaNative - Uses Legacy transfer() function to move native token to treasury contract address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below.

```
function purchaseViaNative(...) external nonReentrant payable{
       ...
        // Calculate Treasury Share of Source Amount ERC20
```

```
uint256 treasuryShareForSourceAmount = calculatePercentAmount(treasuryShareForSourceAmountPercent,msg.value);

        if (treasuryShareForSourceAmount > 0) {

            // Transfer Treasury Share to Treasury

            (bool success) = treasuryContract.call{value:treasuryShareForSourceAmount}("");

            require(success,"transfer to treasury failed");

        }

    }
```

Status

Resolved

[L-10]L1XAppNFTSale#_getLatestNativeQuoteInUSD - chainlink price data could be stale

Description

There is no check if the return value indicates stale data. This could lead to stale prices according to the Chainlink documentation:
The price oracle might return unreliable price data which can lead to a variety of issues in the protocol like pricing of NFT.

Recommendation

Configure a threshold for max time passed since the updated time returned by the oracle. If the prices are older than the threshold window, consider the prices as stale. While chainlink is a reliable source, the protocol should take protection against unexpected scenarios.

Status

Acknowledged

[L-11]ZapContract#_transferToTreasury - Uses Legacy transfer() function to move native token to treasury address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below.

```
function transferToTreasury(
        address _tokenAddress,
        uint256 _amount
    ) external onlyOwner {
    require(treasuryAddress != address(0), "Treasury address is not set");
    if (_tokenAddress == NATIVE_ASSET_ADDRESS) {
        // Send Native to Treasury with reentrancy Protection for Native
        (bool success) = treasuryAddress.call{value:amount}("");
         require(success,"sendFunds failed");
    } else {
        // Send Native to Treasury with reentrancy Protection for ERC20
        IERC20(_tokenAddress).safeTransfer(treasuryAddress, _amount);
    }
}
```

Status

Resolved

[L-12]ZapContract#_initiateZap - Uses Legacy transfer() function to move native token to treasury address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below.

```
function initiateZap(
       ...
    ) external payable nonReentrant {
        ...


        if (_sourceFee > 0) {
            if (_sourceAssetAddress == NATIVE_ASSET_ADDRESS) {
                // Transfer Native asset to this treasury
                payable(treasuryAddress).transfer(_sourceFee);
            } else {
              …
            }
        }
        // Calculate Treasury Share of Source Amount
        uint256 treasuryShareForSourceAmount = calculatePercentAmount(
            treasurySharePercent,
            _sourceAmount
        );
        if (treasuryShareForSourceAmount > 0) {
            // Transfer Treasury Share to Treasury
```

```
            if (_sourceAssetAddress == NATIVE_ASSET_ADDRESS) {
                payable(treasuryAddress).transfer(treasuryShareForSourceAmount);
            } else {
                …
            }
        }
        ...
    }
```

Status

Resolved

[L-13]ZapContract#_executeZap - Uses Legacy transfer() function to move native token to treasury address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below. Replace the `transfer()` function with call instead.

```
function executeZap(
        uint256 amount,
        address receiverAddress,
        address assetAddress
    ) internal {
        …
        if (assetAddress == NATIVE_ASSET_ADDRESS) {
```

```
            // Send Native to Treasury with reentrancy Protection for Native
            payable(receiverAddress).transfer(amount);
        } else {
          …
        }
    }
```

Status

Resolved

[L-14]SwapGasPriceV2Contract#_getChainlinkDataFeedLatestAnswer-chainlink price data could be stale

Description

There is no check if the return value indicates stale data. This could lead to stale prices according to the Chainlink documentation:
The price oracle might return unreliable price data which can lead to a variety of issues in the protocol like pricing of NFT.

Recommendation

Configure a threshold for max time passed since the updated time returned by the oracle. If the prices are older than the threshold window, consider the prices as stale. While chainlink is a reliable source, the protocol should take protection against unexpected scenarios.

Status

Acknowledged

[L-15]SwapV2Contract#_transferToTreasury - Uses Legacy transfer() function to move native token to treasury address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below:

```
function transferToTreasury(
        address _tokenAddress,
        uint256 _amount
    ) external onlyOwner {
  require(treasuryAddress != address(0), "Treasury address is not set");
  if (_tokenAddress == NATIVE_ASSET_ADDRESS) {
     // Send Native to Treasury with reentrancy Protection for Native
     (bool success) = treasuryAddress.call{value:_amount}("");
     require(success,"sendFunds failed");
  } else {
     // Send Native to Treasury with reentrancy Protection for ERC20
     IERC20(_tokenAddress).safeTransfer(treasuryAddress, _amount);
  }
}
```

Status

Resolved

[L-16]SwapV2Contract#_transferToStake - Uses Legacy transfer() function to move native token to STAKE_CONTRACT_ADDRESS address. Another problem in the transferToStake function is the flow of logic.

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below.

Also, refer to the flow of the logic where, first the values read read using the internal Id from `STAKE_CONTRACT_ADDRESS` and later the `STAKE_CONTRACT_ADDRESS` address validated to be non zero address.

Recommendation is to first validate the `STAKE_CONTRACT_ADDRESS` address to be non zero and then make calls on the contract address.

```
function transferToStake(string memory _internalId) external {
      ...
      if (_tokenAddress == NATIVE_ASSET_ADDRESS) {
         // Send Native to Treasury with reentrancy Protection for Native
        (bool success) = STAKE_CONTRACT_ADDRESS.call{value:_amount}("");
        require(success,"sendFunds failed");
      } else {
         // Send Native to Treasury with reentrancy Protection for ERC20
         …
      }
      ...
}
```

Status

Resolved

[L-17]SwapV2Contract#_initiateSwap - Uses Legacy transfer() function to move native token to treasury address and also source native fee

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below.

Status

Resolved

[L-18]SwapV2Contract#_executeSwap - Uses Legacy transfer() function to move native token to receiver address

Description

`transfer()` function is not a recommended way to move native tokens from one account to another due to the 2300 gas limit. The vulnerability may arise due to changes in infrastructure related to gas consumption or even in the receiving contract. If the Tresory address is a smart contract that performs additional operations in the receive or fallback function, the amount of gas may not suffice and revert.

Recommendation

The recommendation is to use `call()` function instead. Please refer to the code snippet below.

Status

Resolved

[L-19]SwapGasPriceV2Contract#_updatePercentFactor - The range of acceptable values for PERCENT_FACTOR should be from 0 to 99.99. The validation is missing

Description

The `PERCENT_FACTOR` is initialised to support from 0 to 99.99% as the valid range of values. But, the `updatePercentFactor` does not enforce the restriction while updating the `PERCENT_FACTOR`. Configuring values out this range will adversely impact the fee computation logic and hence should be prevented.

Recommendation

The recommendation is to add validation in the `updatePercentFactor` function to accept the value of the parameter to be between 0 and 99.99 represented in correct format.

Status

Resolved

[L-20]SwapV2Contract#_SWAP_IN_PROGRESS - Declared variable was never used

Description

`SWAP_IN_PROGRESS` was declared in the contract, but never used.

Recommendation

Remove redundant variables in the contract, if they are not in use.

Status

Resolved

Gas

[G-01] MultiSigWalletSingleBroadcast - Transaction struct could be packed

Description

Packing structs in Solidity could be an effective way to optimise gas costs by reducing the number of storage reads and writes needed to access multiple variables.

```
struct Transaction {
    address to;
    uint256 value;
    uint256 numConfirmations;
    bool executed;
    bytes data;
}
```

In the current implementation, `to`, `value`, `numConfirmations`, and `executed` each object use different slots.

However, moving the `executed` object before the `value` object will save a slot by storing `to` and `executed` objects in the same slot.

Recommendation

Move the `executed` object before the `value` object.

Status

Resolved

QA

[Q-01] MultiSigWalletSingleBroadcast - Floating pragma

Description

The contracts have `pragma solidity ^0.8.0` and it might allow the contracts to be deployed with a different version than the one used for testing.

Different pragma versions being used in test and mainnet may pose unidentified security issues.

Recommendation

Specify a specific version of Solidity in the pragma statement.

Status

Resolved

Centralisation

The LayerOneX project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the LayerOneX project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au